TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
Enero-Junio

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
Patrones de Diseño

TÍTULO ACTIVIDAD:
Actividad de valor en equipo - Patrón de diseño Abstract Factory

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO:

Chávez Moreno Roberto 22210292

Serna Segura Noel Alberto 22210354

Orozco Hernandez Brandon 21212577

Castillejo Robles Lennyn Alejandro 22210880

NOMBRE DEL MAESTRO:

Maribel Guerrero Luis

FECHA:

Miércoles 12 marzo 2026

Introducción

Este proyecto implementa un sistema de gestión de restaurantes utilizando el patrón de diseño **Abstract Factory**. El sistema permite seleccionar diferentes tipos de cocina (China, Japonesa, Mexicana, Italiana, India) y genera un menú completo compuesto por plato fuerte, bebida y postre, donde cada cocina tiene sus propios productos específicos.

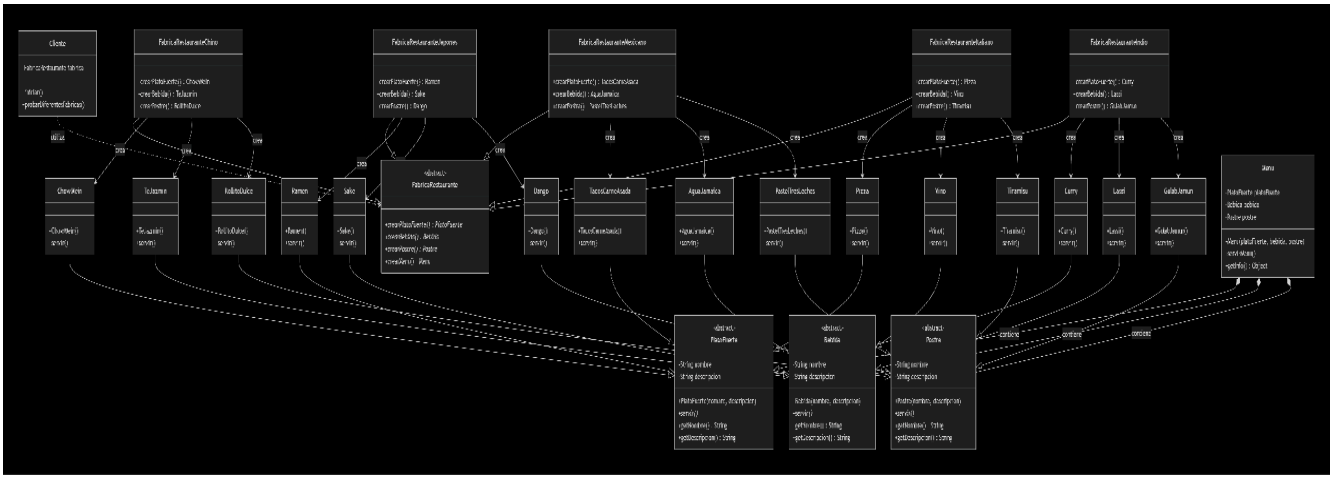
El proyecto cuenta con dos interfaces:

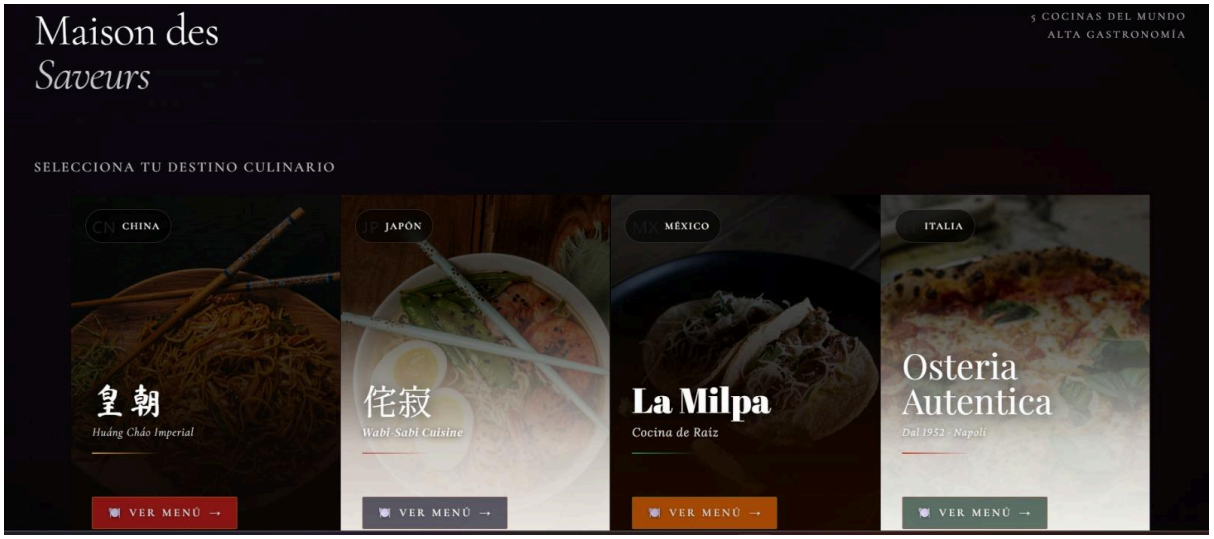
- **Backend:** JavaScript con implementación pura del patrón
- **Frontend:** Interfaz gráfica desarrollada en React que visualiza los menús

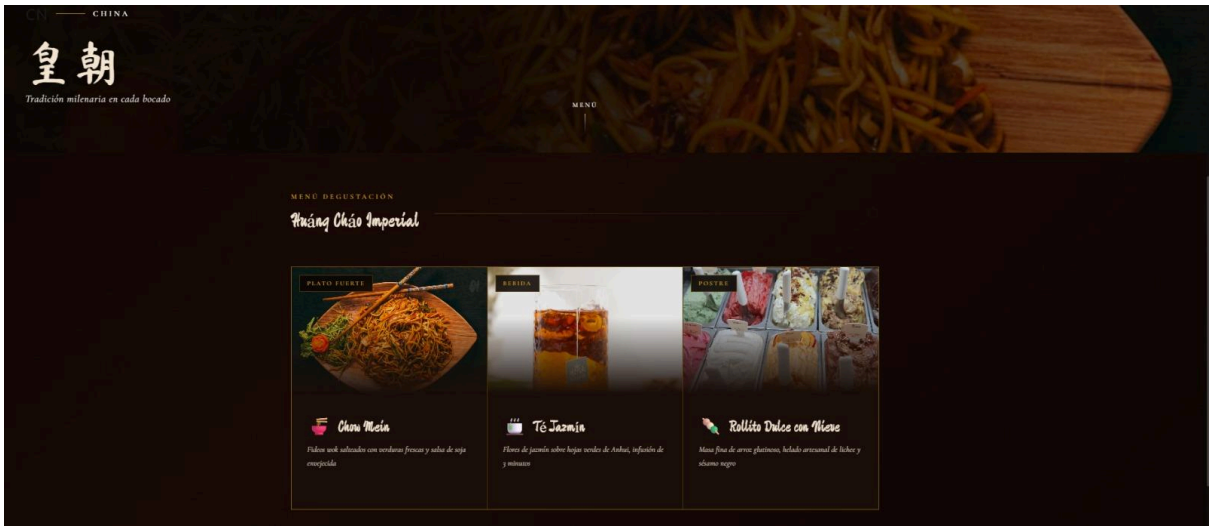
Estructura general:

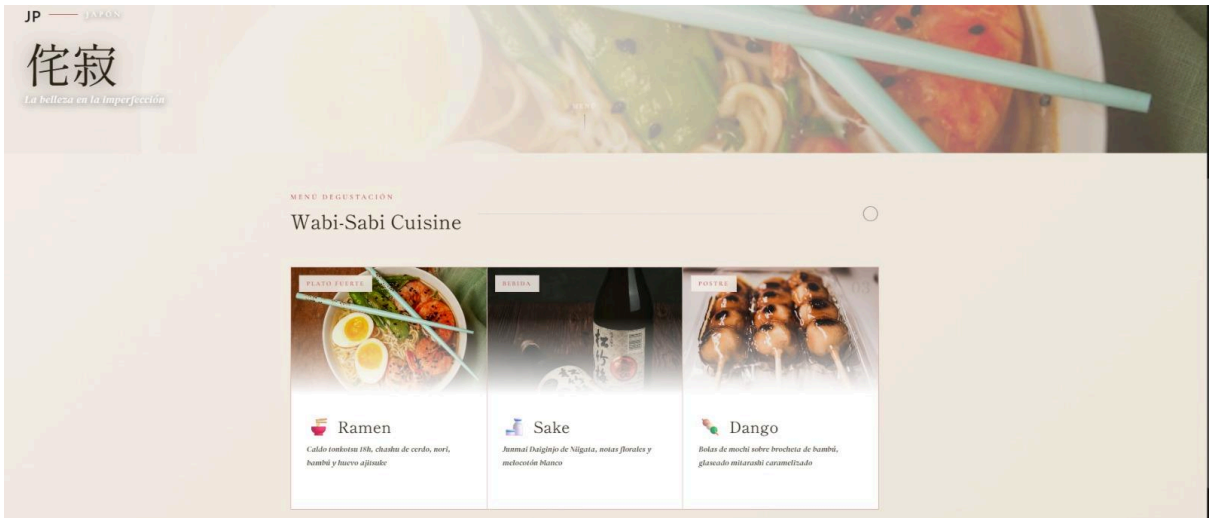
- 5 familias de productos (una por cocina)
- 3 productos por familia (plato fuerte, bebida, postre)
- 15 clases de productos concretos en total
- 5 fábricas concretas (una por cocina)

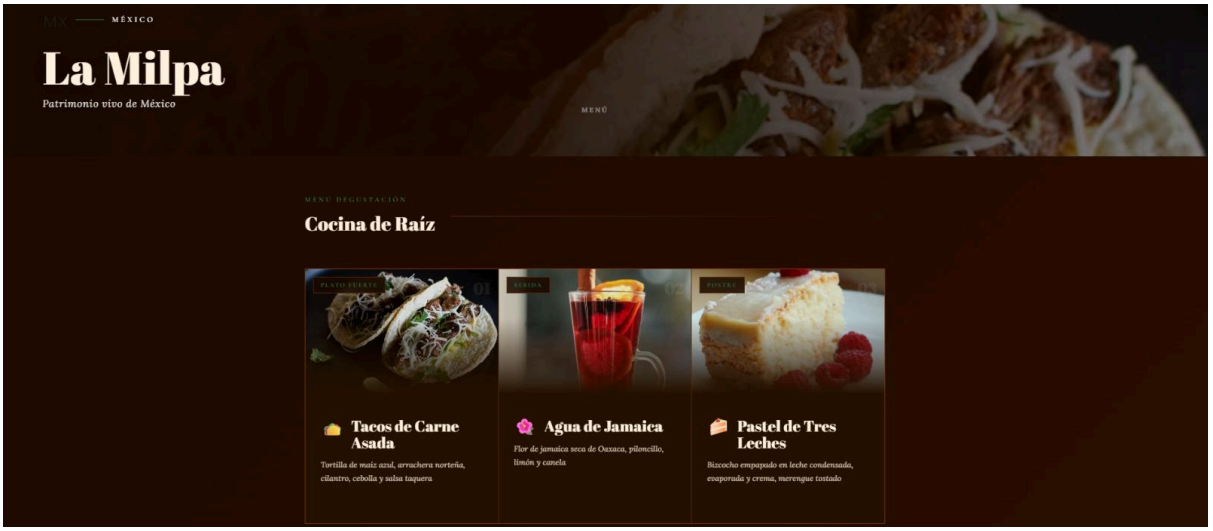
Diagrama UML

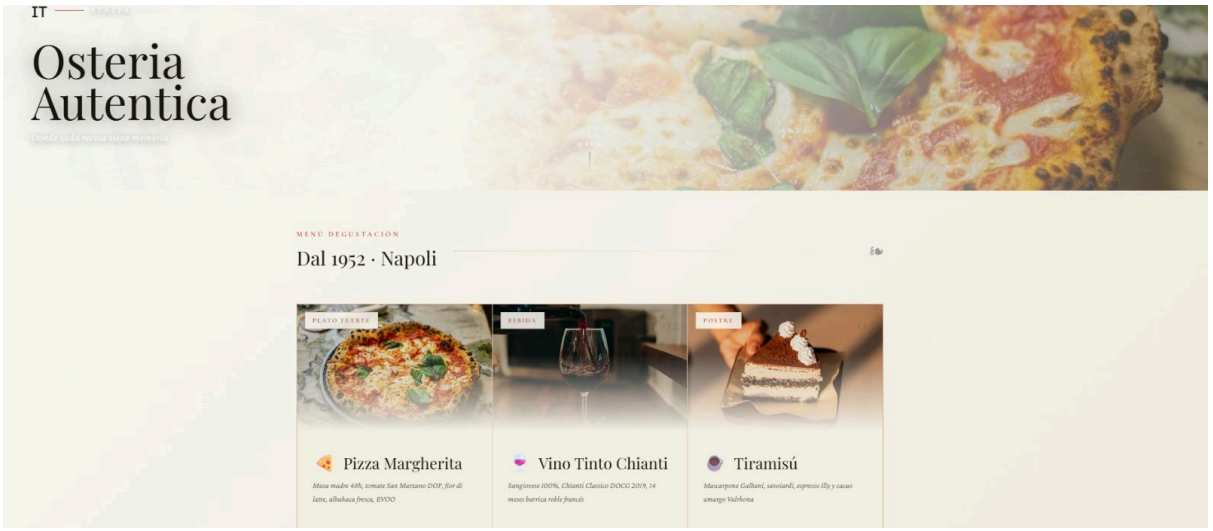


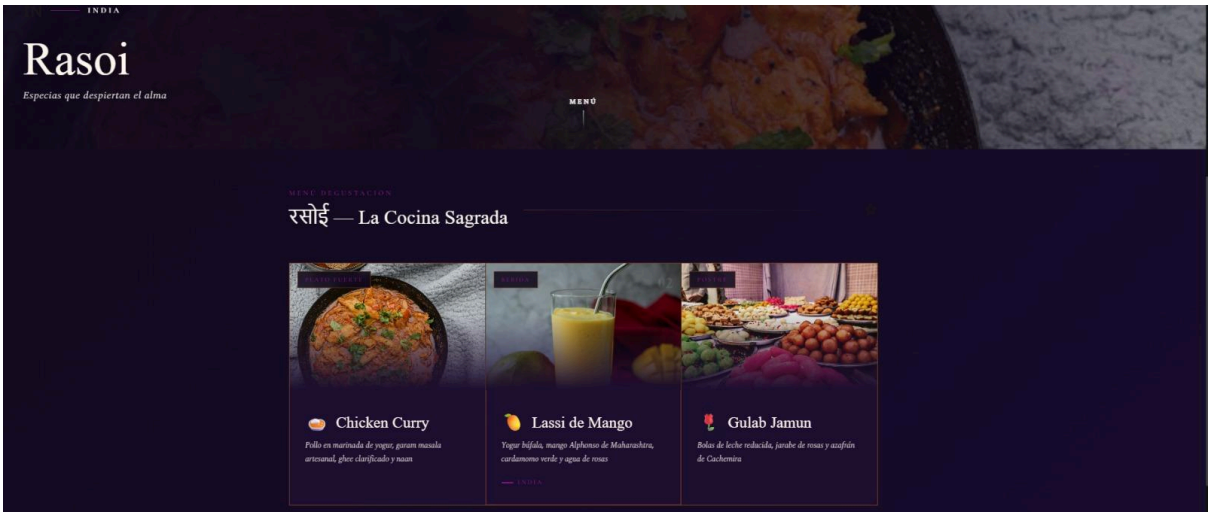

Maison des
Saveurs
5 COCINAS DEL MUNDO
ALTA GASTRONOMÍA
SELECCIONA TU DESTINO CULINARIO
CN CHINA
皇朝
Huáng Cháo Imperial
VER MENÚ →
JP JAPÓN
侘寂
Wabi-Sabi Cuisine
VER MENÚ →
MX MÉXICO
La Milpa
Cocina de Raíz
VER MENÚ →
ITALIA
Osteria
Autentica
Dal 1952 · Napoli
VER MENÚ →

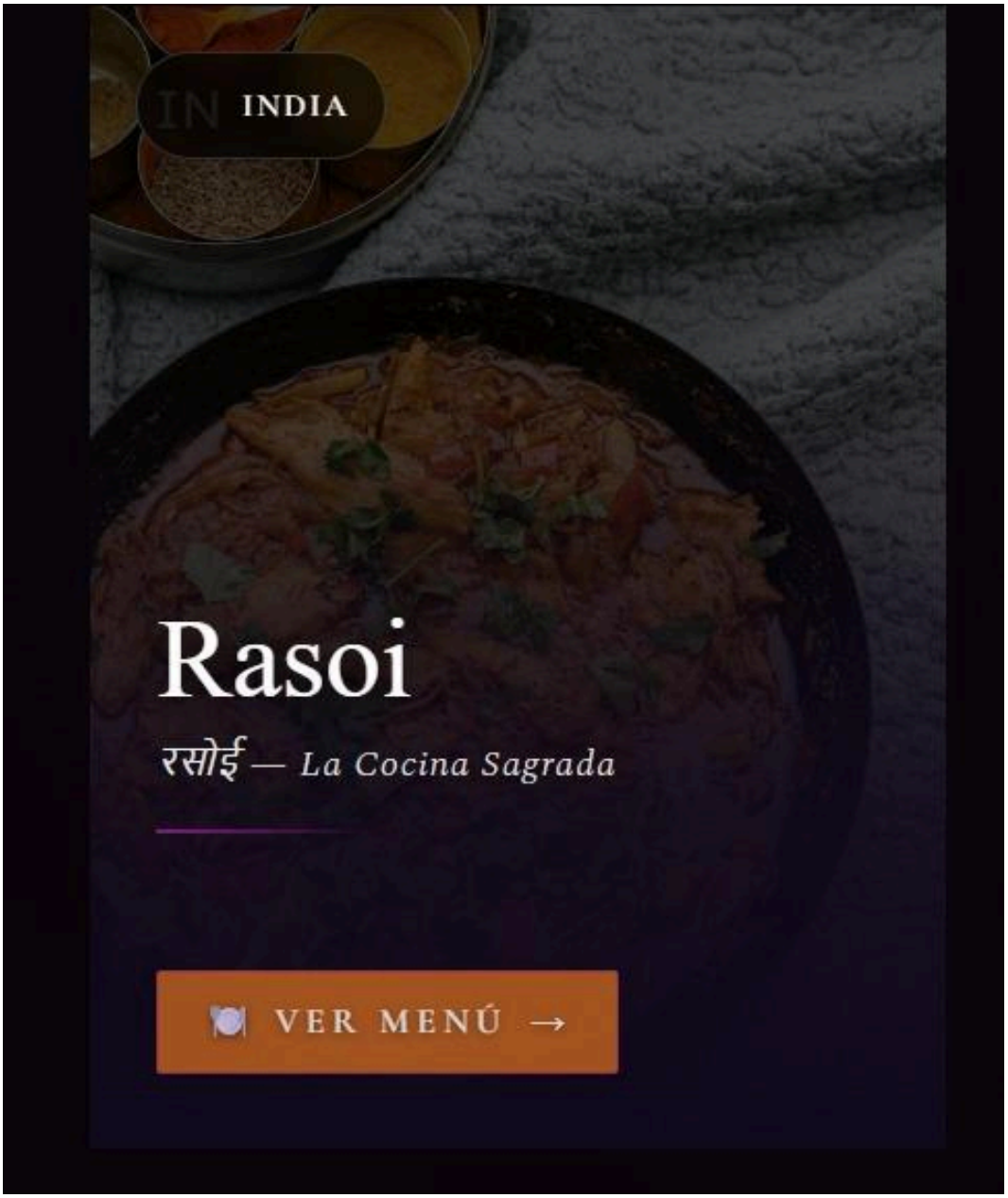

CN CHINA
皇朝
Tradición milenaria en cada bocado
MENÚ
MENÚ DEGUSTACIÓN
Huáng Cháo Imperial
PLATO FUERTE
Chow Mein
Fideos wok salteados con verduras frescas y salsa de soja enseñada
BEBIDA
Té Jazmín
Flores de jazmín sobre hojas verdes de Anhui, infusión de 3 minutos
POSTRE
Rollito Dulce con Miene
Masa fina de arroz glutinoso, helado artesanal de lichee y sésamo negro


JP JAPÓN
侘寂
La belleza en la imperfección
MENÚ
MENÚ DEGUSTACIÓN
Wabi-Sabi Cuisine
PLATO FUERTE
Ramen
Caldo tonkotsu 18h, chashu de cerdo, nori, bambú y huevo ajitsuke
BEBIDA
Sake
Junmai Daiginjo de Niigata, notas florales y melocotón blanco
POSTRE
Dango
Bolas de mochi sobre brocheta de bambú, glaseado mitarashi caramelizado


MX
MÉXICO
La Milpa
Patrimonio vivo de México
MENÚ
MENÚ DEGUSTACIÓN
Cocina de Raíz
PLATO FUERTE
01
BEBIDA
02
POSTRE
03
Tacos de Carne Asada
Tortilla de maíz azul, arrachera norteña, cilantro, cebolla y salsa taquera
Agua de Jamaica
Flor de jamaica seca de Oaxaca, piloncillo, limón y canela
Pastel de Tres Leches
Bizcocho empapado en leche condensada, evaporada y crema, merengue tostado


IT
Osteria Autentica
Donde cada receta tiene memoria
MENÚ
MENÚ DEGUSTACIÓN
Dal 1952 · Napoli
PLATO FUERTE
BEBIDA
POSTRE
Pizza Margherita
Masa madre 48h, tomate San Marzano DOP, fior di latte, albahaca fresca, EVOO
Vino Tinto Chianti
Sangiovese 100%, Chianti Classico DOCG 2019, 14 meses barrica roble francés
Tiramisú
Mascarpone Galbani, savoiardi, espresso Illy y cacao amargo Valrhona


IN
INDIA
Rasoi
Especias que despiertan el alma
MENÚ
MENÚ DEGUSTACIÓN
रसोई — La Cocina Sagrada
PLATO FUERTE
BEBIDA
POSTRE
Chicken Curry
Pollo en marinada de yogur, garam masala artesanal, ghee clarificado y naan
Lassi de Mango
Yogur bífalo, mango Alphonso de Maharashtra, cardamomo verde y agua de rosas
INDIA
Gulab Jamun
Bolas de leche reducida, jarabe de rosas y azafrán de Cachemira



Explicación

¿Qué es el patrón Abstract Factory?

El patrón de fábrica abstracta en ingeniería de software es un patrón de diseño que permite crear familias de objetos relacionados sin imponer sus clases concretas, encapsulando un grupo de fábricas individuales con un tema común sin especificar sus clases concretas.

En este proyecto:

- **Familia de productos:** Plato fuerte, Bebida, Postre
- **Variantes de la familia:** Cocina China, Japonesa, Mexicana, Italiana, India
- **Cliente:** Trabaja con interfaces, no con implementaciones concretas

Componentes del patrón en el proyecto

Componente	Clase en el proyecto	Responsabilidad
Producto Abstracto	PlatoFuerte, Bebida, Postre	Define la interfaz común para todos los productos de su tipo
Producto Concreto	ChowMein, Ramen, Tacos, etc.	Implementa la interfaz del producto abstracto con comportamiento específico
Fábrica Abstracta	FabricaRestaurante	Declara los métodos para crear cada producto de la familia
Fábrica Concreta	FabricaRestauranteChino, etc.	Implementa la creación de productos para una variante específica
Cliente	Cliente (backend), App (frontend)	Usa solo las interfaces abstractas; no conoce las clases concretas

¿Cómo se evita el uso excesivo de NEW?

Problema sin el patrón

```
// El código cliente está lleno de NEW dispersos
function crearMenu(tipo) {
 if (tipo === "chino") {
   const plato = new ChowMein();       // ← NEW
   const bebida = new TeJazmin();      // ← NEW
   const postre = new RollitoDulce(); // ← NEW
   return { plato, bebida, postre };
 } else if (tipo === "japones") {
```

```
    const plato = new Ramen();          // ← NEW
    const bebida = new Sake();          // ← NEW
    const postre = new Dango();         // ← NEW
    return { plato, bebida, postre };
  }
  // ... y así para cada restaurante
}
// Total: 15 NEW distribuidos en múltiples lugares
```

Solución con Abstract Factory

```
// Los NEW están encapsulados en las fábricas concretas
export class FabricaRestauranteChino extends FabricaRestaurante {
  crearPlatoFuerte() { return new ChowMein(); }      // ← NEW aquí
  crearBebida() { return new TeJazmin(); }           // ← NEW aquí
  crearPostre() { return new RollitoDulce(); }       // ← NEW aquí
}

// El cliente solo usa un NEW para la fábrica
const fabrica = new FabricaRestauranteChino();       // ← Único
NEW en cliente
const menu = fabrica.crearMenu();                    // ← Sin NEW

// crearMenu() llama a los métodos que contienen los NEW
```

Beneficios

- **Centralización:** Todos los new están en 5 lugares fijos (las fábricas concretas)
- **Desacoplamiento:** El cliente no sabe qué clases concretas se están instanciando
- **Mantenibilidad:** Si cambia el constructor de ChowMein, solo se modifica en un lugar

¿Cómo agregar una nueva cocina?

Para agregar una nueva cocina al sistema, debes seguir exactamente la misma estructura que ya tienen las cocinas existentes.

Estructura de la Cocina China (Ejemplo existente)

Paso 1: Crear los productos concretos

backend/src/china/ChowMein.js (Plato fuerte)

```
import { PlatoFuerte } from '../PlatoFuerte.js';

export class ChowMein extends PlatoFuerte {
  constructor() {
    super("Chow Mein", "Fideos chinos salteados con verduras y
salsa de soja");
  }

  servir() {
    console.log(" Sirviendo Chow Mein caliente con palillos
chinos");
  }
}
```

backend/src/china/TeJazmin.js (Bebida)

```
import { Bebida } from '../Bebida.js';

export class TeJazmin extends Bebida {
  constructor() {
    super("Té Jazmín", "Té verde aromatizado con flores de
jazmín");
  }

  servir() {
    console.log(" Sirviendo Té Jazmín caliente en taza de
porcelana");
  }
}
```

backend/src/china/RollitoDulce.js (Postre)

```
import { Postre } from '../Postre.js';

export class RollitoDulce extends Postre {
  constructor() {
    super("Rollito Dulce con Nieve", "Rollito de primavera dulce
relleno de crema helada");
  }

  servir() {
    console.log(" Sirviendo Rollito Dulce con nieve y sirope");
  }
}
```

Paso 2: Crear la fábrica concreta

backend/src/fabricas/FabricaRestauranteChino.js

```
import { FabricaRestaurante } from '../FabricaRestaurante.js';
import { ChowMein } from '../china/ChowMein.js';
import { TeJazmin } from '../china/TeJazmin.js';
import { RollitoDulce } from '../china/RollitoDulce.js';

export class FabricaRestauranteChino extends FabricaRestaurante {
  crearPlatoFuerte() {
    return new ChowMein();
  }

  crearBebida() {
    return new TeJazmin();
  }

  crearPostre() {
    return new RollitoDulce();
  }
}
```

Paso 3: Registrar en el cliente

backend/src/Cliente.js (fragmento)

```
// Importar la fábrica
import { FabricaRestauranteChino } from
'./fabricas/FabricaRestauranteChino.js';

// En el switch del método iniciar()
switch (opcion) {
  case "1":
    fabrica = new FabricaRestauranteChino();
    nombreRestaurante = " RESTAURANTE CHINO SELECCIONADO";
    break;
  // otros casos...
}

// En el menú de opciones
const opcion = await this.pregunta(
  "Seleccione el tipo de restaurante:\n" +
  "1. Chino \n" +  // Opción para China
  "2. Japonés \n" +
  "3. Mexicano \n" +
  "4. Italiano \n" +
  "5. Indio \n" +
  "Ingrese su opción (1-5): "
);
```

Instrucciones para agregar una NUEVA cocina:

1. **Crea una nueva carpeta** con el nombre de tu cocina (ej: thai/, francesa/, árabe/)
2. **Dentro de esa carpeta, crea 3 archivos:**
   - TuPlatoFuerte.js (hereda de PlatoFuerte)
   - TuBebida.js (hereda de Bebida)
   - TuPostre.js (hereda de Postre)
3. **Crea un archivo de fábrica** llamado FabricaRestauranteTuCocina.js
4. **En la fábrica**, implementa los 3 métodos para que devuelvan instancias de tus nuevos productos
5. **En el cliente** (Cliente.js):
   - Importa tu nueva fábrica
   - Agrega un nuevo case en el switch
   - Agrega una nueva opción en el menú de selección

Conclusiones

En este proyecto, como equipo, implementamos el patrón Abstract Factory simulando un sistema de restaurantes con cinco tipos de cocina diferentes: China, Japonesa, Mexicana, Italiana e India. Lo más valioso que aprendimos es que este patrón nos ayudó a organizar mejor nuestro código porque todas las instrucciones para crear los platos, bebidas y postres quedaron encapsuladas dentro de las fábricas y esto facilitó mucho el trabajo en equipo. También descubrimos que si en el futuro queremos agregar un nuevo restaurante, por ejemplo de cocina Thai, solo tendríamos que crear tres archivos nuevos para sus productos y una nueva fábrica, sin tener que modificar todo el código. El proyecto nos sirvió para entender que cuando se planea bien la estructura desde el principio, el mantenimiento y la escalabilidad son mucho más sencillos. Además, comprobamos que la misma lógica del patrón funcionó tanto en el programa de consola que hicimos como en la página web que desarrollamos con React, lo que demuestra la versatilidad de este diseño para proyectos más grandes y complejos.

Referencias

Abstract factory pattern.
https://en.wikipedia.org/wiki/Abstract_factory_pattern

Abstract Factory.
https://refactoring.guru/es/design-patterns/abstract-factory

Mermaid.js - Class Diagram Syntax.
https://mermaid.js.org/syntax/classDiagram.html